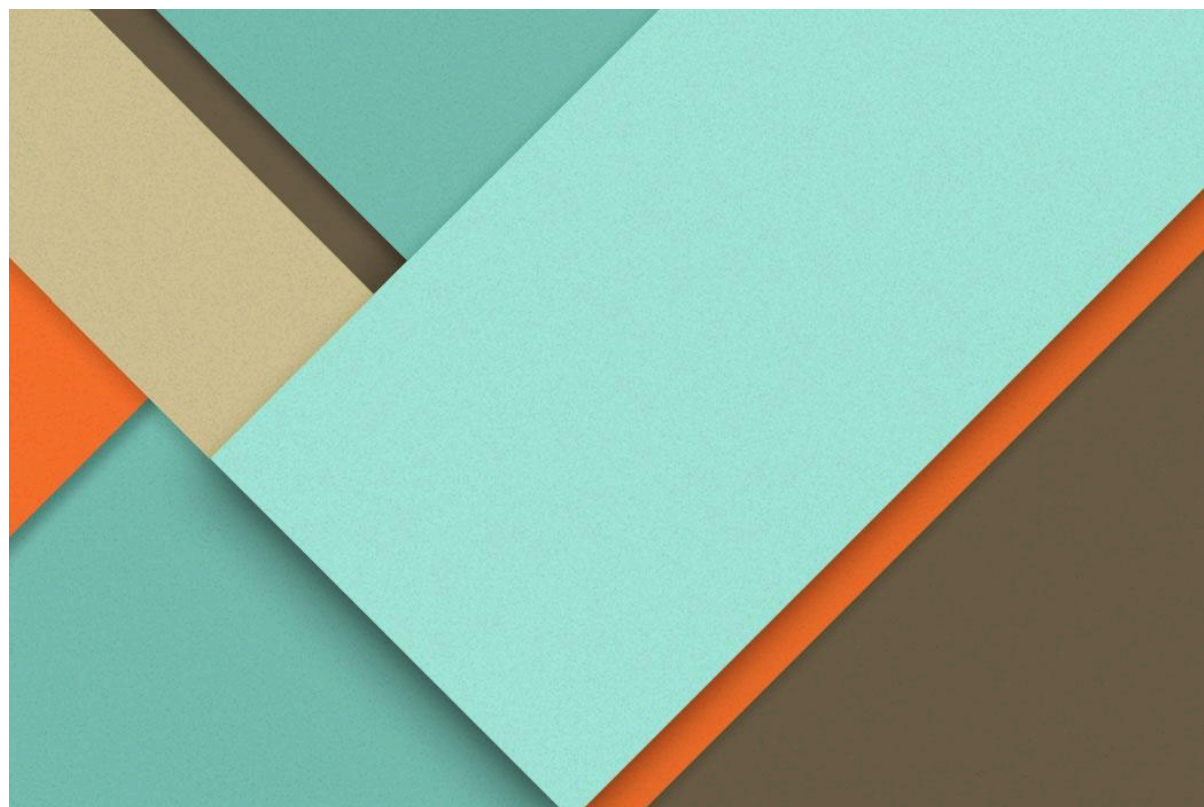




Ingeniería de Software (3304)

Taller Actividad N°4

Integrantes

Pautasso Gisel

Pautasso Stefani

Rodeghiero Tomás



Análisis de RuboCop

La herramienta RuboCop ayudó a identificar un total de **820 ofensas** en el código, distribuidas en **59 archivos**. Gracias a esto, fue posible detectar y corregir varios problemas, logrando que el código sea más limpio y fácil de mantener. Además, la experiencia nos permitió incorporar mejores prácticas de programación en Ruby.

Clasificación de Ofensas

A continuación se detalla la clasificación de las ofensas encontradas:

- **Style/FrozenStringLiteralComment:** 59
- **Bundler/OrderedGems:** 5
- **Style/Documentation:** 22
- **Layout/TrailingWhitespace:** 255
- **Style/SymbolArray:** 1
- **Metrics/BlockLength:** 18
- **Style/NumericLiterals:** 1
- **Style/StringLiterals:** 121
- **Style/WordArray:** 1
- **Layout/IndentationWidth:** 3
- **Layout/BlockAlignment:** 3
- **Layout/CommentIndentation:** 18
- **Layout/LeadingCommentSpace:** 19
- **Layout/LineLength:** 220
- **Lint/UselessAssignment:** 7
- **Layout/TrailingEmptyLines:** 21
- **Layout/MultilineMethodCallIndentation:** 13
- **Style/RedundantSelf:** 2
- **Layout/EmptyLinesAroundClassBody:** 5
- **Style/HashSyntax:** 2
- **Lint/UnusedBlockArgument:** 2
- **Metrics/ClassLength:** 1
- **Style/IfUnlessModifier:** 3
- **Style/IfInsideElse:** 1
- **Lint/EmptyConditionalBody:** 2
- **Layout/EmptyLinesAroundBlockBody:** 11
- **Layout/IndentationConsistency:** 1

- **Layout/SpaceAfterColon:** 1
- **Style/RedundantFileExtensionInRequire:** 1
- **Style/BlockComments:** 1

Bad Smells in Code - Capítulo 3

Al analizar la lista de problemas identificados por RuboCop pudimos relacionarla con los *code smells* descritos en el capítulo 3 de la bibliografía sugerida:

1. **Style/FrozenStringLiteralComment, Style/Documentation y Style/RedundantFileExtensionInRequire/ Style/NumericLiterals y Style/StringLiterals/Style/WordArray y Style/SymbolArray**

El uso excesivo de literales de tipo *String*, como en las categorías de lecciones ('identify', 'preventive', 'action'), sugiere **Obsesión por Primitivos**, ya que depender de cadenas sueltas sin un contexto claro hace que el código sea menos legible y más propenso a errores. Con la directiva `# frozen_string_literal: true` se busca prevenir modificaciones accidentales de estos valores al garantizar su inmutabilidad, lo que mejora la integridad de los datos y la claridad del código.

2. **Layout/TrailingWhitespace, Layout/TrailingEmptyLines, Layout/IndentationWidth, Layout/BlockAlignment, Layout/CommentIndentation, Layout/LeadingCommentSpace y Layout/EmptyLinesAroundClassBody / Layout/EmptyLinesAroundBlockBody, Layout/IndentationConsistency, Layout/MultilineMethodCallIndentation y Layout/SpaceAfterColon**

Estos problemas de alineación y espaciado corresponden a **Inconsistencia de Estilo**.

3. **Layout/LineLength:**

Este problema se relaciona con el uso excesivo de texto en las *seeds* del proyecto, donde las líneas largas dificultan la legibilidad y el mantenimiento del código. En el libro, se menciona que los **Métodos Largos o Clases Grandes** son difíciles de seguir, lo que sugiere que el código se podría beneficiar de la refactorización al dividir líneas largas en fragmentos más manejables o encapsular datos significativos en estructuras adecuadas.

4. **Lint/UselessAssignment y Lint/UnusedBlockArgument:**

Estos son ejemplos de **Código Redundante o Especulación General**, donde variables o argumentos están definidos pero nunca usados. Esto indica que podría haber lógica sin

valor, lo que dificulta el mantenimiento y la claridad, al introducir confusión sobre la intención del código.

5. Metrics/BlockLength y Metrics/ClassLength:

Este es un problema de *Método Largo* o *Clase Grande* (server.rb)

6. Style/IfUnlessModifier y Style/IfInsideElse / Lint/EmptyConditionalBody:

Esto indica una estructura condicional compleja o confusa, *Cadenas de Mensajes* o *Condiciones Anidadas*, muestra que hay estructuras de decisión difíciles de leer o innecesariamente complejas.

7. Style/BlockComments:

Esto indica un problema de *Comentarios*.

Los Code Smells más relevantes son *Inconsistencia de Estilo, Método Largo, Clase Grande, Obsesión por Primitivos, Especulación General, y Comentarios*.

Estadísticas Generales

Resumen de las ofensas que se presentaron con mayor frecuencia en el proyecto:

1. **Layout/TrailingWhitespace**: 255 ofensas
2. **Style/StringLiterals**: 121 ofensas
3. **Style/FrozenStringLiteralComment**: 59 ofensas

Es importante destacar la ofensa **Metrics/ClassLength**, que, aunque apareció solo una vez, representa una advertencia significativa en términos de diseño de código. Esta ofensa indicaba que la longitud de una de nuestras clases excedía el límite recomendado, lo que nos llevó a refactorizar y dividir su funcionalidad en controladores dedicados para mejorar la organización y mantenibilidad del proyecto

Distribución de las ofensas en los archivos:

Particularmente, se observó que la mayoría de las ofensas fueron generadas en el archivo de semillas (**db/seeds**), donde se introdujo una gran cantidad de texto explicativo, así como preguntas y opciones, lo que resultó en un número significativo de ofensas de tipo **Layout** y **Style**. Las ofensas se distribuyeron en los siguientes archivos:

- **db/seeds/options.rb**: 219 ofensas
- **db/seeds/questions.rb**: 69 ofensas
- **db/seeds/lessons.rb**: 117 ofensas
- **db/seeds/tests.rb**: 6 ofensas
- **db/seeds/sections.rb**: 1 ofensa

Además, se identificó que el archivo **server.rb** presentaba un tamaño considerable de clase, lo que llevó a una ofensa de **Metrics/ClassLength**. Esta clase fue posteriormente refactorizada en **app.rb** y dividida en controladores para mejorar la organización del código, como se explica más adelante en la sección de refactorización.

Ofensas No Autocorregibles

Algunas ofensas requerían intervención manual, y estas son las que se identificaron:

- **Style/FrozenStringLiteralComment:** Se detectó la necesidad de agregar un comentario especial al principio de cada archivo (`# frozen_string_literal: true`) para hacer que las cadenas sean inmutables (congeladas). Esto ayuda a mejorar el rendimiento al evitar que Ruby cree nuevos objetos de cadena cada vez que se usa el mismo literal. La solución implicó revisar cada archivo para asegurar que esta opción no afectará cadenas que necesitaban ser modificables.
- **Bundler/OrderedGems:** Se encontró que las gemas en el `Gemfile` no estaban ordenadas correctamente, lo que requirió ajustes manuales para asegurar el orden adecuado.
- **Style/Documentation:** Se identificó la falta de documentación adecuada en varias partes del código. Esto se abordó mediante la implementación de comentarios y descripciones para mejorar la claridad y comprensión del código.
- **Metrics/ClassLength:** Clases más extensas de lo recomendado. Esto se observó en la clase `App`, que posteriormente se refactorizó en partes más manejables para facilitar su mantenimiento y comprensión.
- **Lint/UselessAssignment:** Se detectaron asignaciones de variables innecesarias que no aportaban valor. Se eliminaron para mantener el código limpio y eficiente.
- **Style/RedundantSelf:** En ciertos contextos, el uso de `self` era redundante. Se revisaron estos casos manualmente para simplificar el código donde fuera posible.

Conclusión del análisis con RuboCop

El análisis realizado con RuboCop ha sido fundamental para identificar y corregir un total de 820 ofensas en nuestro código. Gracias a su capacidad de autocorrección, 779 ofensas fueron solucionadas automáticamente.

Entre las ofensas detectadas, se encontró una violación de **Metrics/ClassLength** en la clase principal **App**, que manejaba todas las rutas de la aplicación. Esto llevó a una refactorización significativa, en la que el archivo **server.rb** fue dividido en **app.rb** y controladores específicos, como se explica en la sección de Refactorización más adelante. Por otro lado, la mayoría de las ofensas comunes aparecieron en el manejo de las semillas de la base de datos debido a la gran cantidad de texto en el código, lo cual generó problemas de formato y longitud de línea.

Al abordar las ofensas no autocorregibles, se mejoró la calidad del código y se implementaron mejores prácticas, lo que asegura un código más limpio, mantenible y fácil de leer.

Refactorización: Clase “server.rb” a “app.rb” en Controladores

Contexto de la Refactorización

En el contexto del proyecto, se identificó que la clase **server** original presentaba ciertas limitaciones en cuanto a su diseño, ya que centralizaba múltiples responsabilidades. Esta arquitectura, aunque funcional, generaba problemas de escalabilidad, dificultaba el mantenimiento y aumentaba la complejidad para agregar nuevas funcionalidades. Por este motivo, se decidió implementar una refactorización de la clase **server** a **app**, aplicando principios de diseño de software para distribuir la lógica en controladores específicos. A continuación se detallan los objetivos, el proceso de refactorización, los cambios implementados y los beneficios alcanzados.

Objetivo de la Refactorización

El propósito principal de la refactorización de la clase **app** fue abordar problemas derivados de una arquitectura monolítica en la que toda la lógica estaba concentrada en una única clase. La clase **server** originalmente gestionaba todas las rutas y la lógica asociada, lo que se tradujo en una clase con múltiples responsabilidades. Los objetivos específicos de la refactorización fueron los siguientes:

- **Separar responsabilidades:** Asignar cada conjunto de rutas y funcionalidades a controladores individuales, de forma que cada módulo específico del proyecto tuviera su propio controlador.

- **Facilitar el mantenimiento:** Organizar el código de manera que los cambios y mejoras futuros sean más localizados y menos propensos a impactar otros módulos.
- **Aumentar la escalabilidad:** Crear una estructura modular que permita agregar nuevas funcionalidades sin incrementar significativamente la complejidad en una sola clase.

Proceso de Refactorización

Para llevar a cabo esta refactorización, se dividió la clase server en varios controladores, cada uno responsable de una funcionalidad o módulo específico. A continuación se detallan los pasos realizados en el proceso:

1. **Identificación de Rutas y Funcionalidades:** Se revisó el código de la clase app para identificar las rutas y funciones principales. A partir de esta revisión, se categorizaron las funcionalidades en módulos independientes.
2. **Creación de Controladores Específicos:** Se crearon los siguientes controladores, cada uno encapsulando un grupo de rutas relacionadas:
 - **MainController:** Encargado de gestionar las rutas principales del proyecto.
 - **AuthController:** Responsable de toda la lógica de autenticación de usuarios, incluyendo inicio de sesión y verificación de permisos.
 - **AdminController:** Controlador específico para las rutas y operaciones administrativas.
 - **SectionsController:** Maneja las secciones específicas del contenido.
 - **ProfileController:** Gestiona las rutas relacionadas con el perfil del usuario, incluyendo la visualización y edición de perfil.
 - **RankingController:** Controlador dedicado a las operaciones y rutas relacionadas con el ranking de usuarios.
3. **Migración de Métodos a Controladores:** Una vez creados los controladores, se migraron los métodos y funciones específicas de cada módulo desde server hacia su controlador correspondiente. Por ejemplo, las funciones de autenticación fueron trasladadas al AuthController, y las funciones relacionadas con el perfil de usuario fueron movidas al **ProfileController**.
4. **Modificación de la Configuración de Rutas:** Se actualizó la configuración de rutas en la clase app para redirigir cada conjunto de rutas al controlador adecuado. Esto permitió que cada controlador gestionara únicamente sus propias rutas, promoviendo así la independencia de cada módulo.

Cambios Específicos en los Métodos

Cada controlador ahora contiene métodos específicos relacionados con su responsabilidad. Esto permitió una mayor cohesión y claridad en el propósito de cada método. A continuación se muestran ejemplos de cambios en los métodos:

- **Métodos de Autenticación:** Los métodos encargados de gestionar la autenticación y verificación de permisos fueron trasladados a **AuthController**.
- **Métodos de Perfil:** Las funciones relacionadas con la visualización y edición del perfil de usuario se migraron al **ProfileController**.
- **Métodos de Administración:** La lógica administrativa fue aislada en el **AdminController**, permitiendo un mejor control y organización de las operaciones administrativas.

Esta refactorización se alinea directamente con la técnica de **"Extract Method"** que describe Martin Fowler. Según esta técnica, es crucial tomar porciones de código que tienen una responsabilidad específica y moverlas a métodos independientes. Esto no solo mejora la legibilidad del código, sino que también facilita su mantenimiento y prueba.

Impacto y Beneficios de la Refactorización

La refactorización de la clase app trajo consigo numerosos beneficios, tanto en términos de estructura como de mantenibilidad:

- **Modularidad y Mantenimiento:** La separación en controladores mejoró la modularidad del código, haciendo que cada sección del sistema esté aislada en su propio controlador. Esto permite que los cambios en un módulo específico no afecten a otros módulos.
- **Escalabilidad:** Con la lógica distribuida en múltiples controladores, se facilita la incorporación de nuevas funcionalidades. Agregar rutas o métodos adicionales se realiza ahora de manera más localizada en el controlador correspondiente, sin necesidad de modificar una única clase centralizada.
- **Claridad y Legibilidad:** La división del código en controladores mejoró la legibilidad del proyecto. Cada controlador tiene un propósito claro y está limitado a una funcionalidad específica, lo que reduce la carga cognitiva al leer y modificar el código.

Consideraciones para Futuros Cambios

Esta nueva estructura modular facilita futuros desarrollos, incluyendo la implementación de pruebas unitarias. La configuración actual permite:

- **Pruebas Unitarias:** Cada controlador puede ser testeado de manera independiente, lo que simplifica la verificación de funcionalidades específicas y asegura que cada módulo funcione correctamente antes de su integración.
- **Implementación de Middlewares:** En esta arquitectura modular, se pueden implementar middlewares en cada controlador de forma individual, permitiendo añadir lógica adicional, como validación de permisos o auditoría, sin afectar otros módulos.

Conclusión de la Refactorización

La refactorización de la clase server a **app** en el proyecto del taller supuso un avance significativo en la organización del código. Al aplicar principios de separación de responsabilidades, se logró una estructura modular y escalable, con beneficios claros en términos de mantenibilidad y claridad. La división en controladores específicos permite un control más preciso de cada funcionalidad, facilitando el desarrollo y el mantenimiento a largo plazo. Esta refactorización representa una base sólida para futuros desarrollos y mejoras en el proyecto.